

FinPredict

Intelligent Loan Approval System

MLOps Project Report

Name : Md Aminul Islam Rony

Roll : ID25M804

Domain: Financial Technology (FinTech) | Predictive Analytics

Technology Stack: Python | FastAPI | React | XGBoost | MLflow | Airflow | DVC | Docker | Prometheus | Grafana

April 2026

1. Introduction and Project Objectives

1.1 Problem Statement

Manual loan approval processes in financial institutions are slow, inconsistent, and prone to human bias. Loan officers must evaluate dozens of variables — credit scores, income levels, debt ratios, employment history — and make decisions that have significant financial consequences for both the institution and the applicant.

FinPredict addresses this problem by building an AI-powered loan approval prediction system that automates this decision-making process using machine learning, delivering instant, consistent, and data-driven predictions.

1.2 Project Objectives

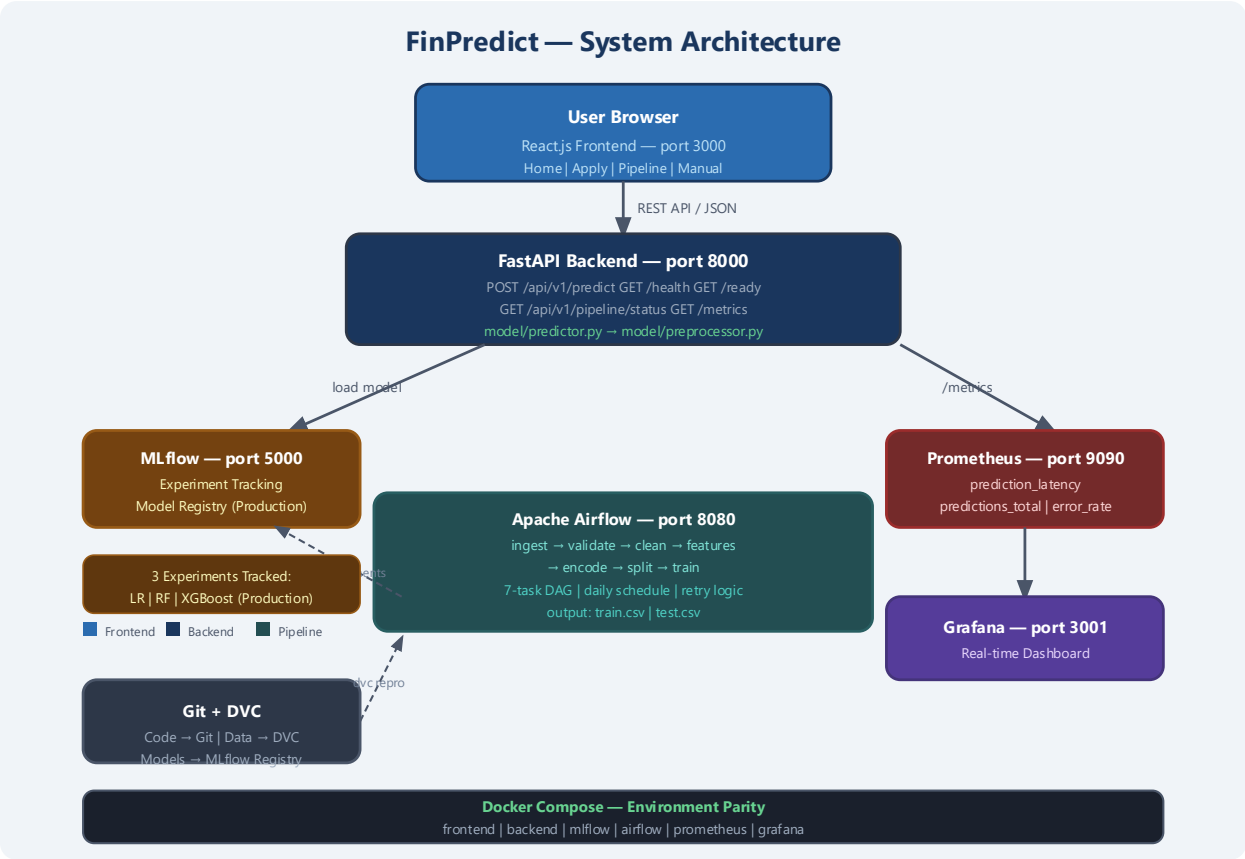
- Build an end-to-end AI application that predicts loan approval or rejection based on applicant financial data
- Implement a full MLOps lifecycle covering data ingestion, preprocessing, model training, deployment, and monitoring
- Expose the model as a REST API (FastAPI) consumed by a React.js web frontend
- Ensure strict loose coupling between frontend and backend — connected only via configurable REST API calls
- Implement experiment tracking (MLflow), data versioning (DVC), pipeline orchestration (Airflow), and observability (Prometheus + Grafana)
- Containerize all services using Docker and Docker Compose for environment parity

1.3 Business and ML Metrics

Metric Type	Metric	Target	Achieved
ML Metric	F1-Score	> 0.85	0.8884
ML Metric	ROC-AUC	> 0.85	0.8188
ML Metric	Accuracy	> 0.80	0.8502
Business Metric	Inference Latency	< 200ms	< 50ms
Business Metric	Pipeline Throughput	> 1000 records/s	~3,900 records/s
Business Metric	System Uptime	> 99%	Docker healthcheck

1.4 System Architecture Overview

FinPredict follows a microservices architecture with six independent Docker containers communicating over a shared Docker network (finpredict-net). The frontend communicates with the backend exclusively via REST API calls, satisfying the loose coupling requirement.



Component	Technology	Port	Role
Frontend	React.js	3002	User interface for loan applications
Backend	FastAPI + Python	8000	REST API + model inference engine
Model Registry	MLflow	5000	Experiment tracking + model serving
Pipeline Orchestrator	Apache Airflow	8080	7-task data pipeline DAG
Metrics Scraper	Prometheus	9090	Instrumentation and alerting
Dashboard	Grafana	3001	Real-time metrics visualization

2. Data Engineering and Model Architecture

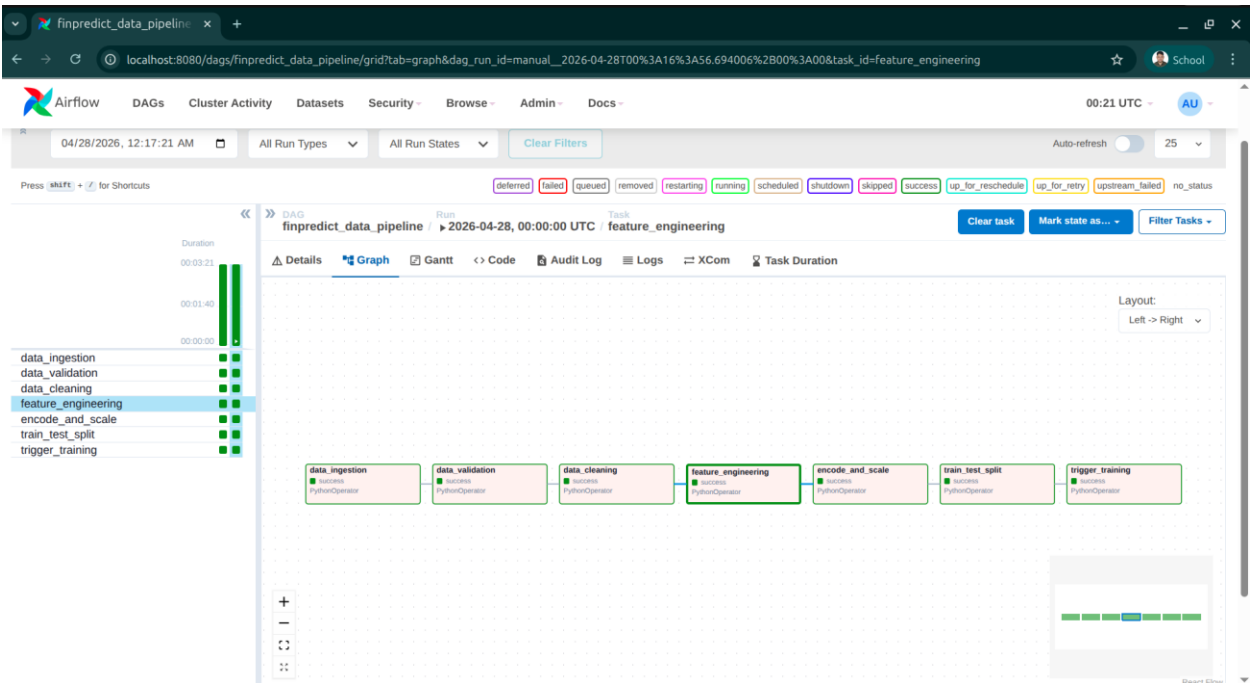
2.1 Dataset

The project uses a structured loan dataset containing 52,000 records and 27 features representing real-world loan application data from a financial institution.

Property	Value
Total Records	52,000 rows
Features	27 columns
Target Variable	Loan_Approval_Status (0 = Rejected, 1 = Approved)
Class Distribution	64.2% Approved, 35.8% Rejected
Missing Values	None
Duplicate Rows	None
Key Features	Credit_Score, Annual_Income, Outstanding_Debt, Loan_Amount_Requested

2.2 Airflow Data Pipeline — 7-Task DAG

The data engineering pipeline is implemented as an Apache Airflow DAG with 7 sequential tasks. It runs on a daily schedule and includes retry logic, failure tracking, and a visual management console.



Task	Function	Input	Output
data_ingestion	Load raw CSV, validate row count	Loan_Dataset.csv	staged.csv
data_validation	Schema checks, null detection, target validation	staged.csv	Validation log
data_cleaning	Rename columns, drop unused cols, clip outliers	staged.csv	cleaned.csv
feature_engineering	Derive 3 new ratio features, save baselines	cleaned.csv	featured.csv
encode_and_scale	Binary/ordinal/one-hot encoding, StandardScaler	featured.csv	encoded.csv + scaler.pkl
train_test_split	80/20 stratified split	encoded.csv	train.csv + test.csv
trigger_training	Train XGBoost, log to MLflow	train.csv + test.csv	Registered model

2.3 Feature Engineering

Three new features were engineered from existing columns to improve model performance:

- $\text{Debt_to_Income_Ratio} = \text{Outstanding_Debt} / \text{Annual_Income}$
- $\text{Loan_to_Income_Ratio} = \text{Loan_Amount_Requested} / \text{Annual_Income}$
- $\text{Expense_to_Income_Ratio} = (\text{Monthly_Expenses} \times 12) / \text{Annual_Income}$

2.4 Pipeline Performance

Stage	Duration	Records
data_ingestion	~0.8s	52,000
data_validation	~0.5s	52,000
data_cleaning	~1.2s	52,000
feature_engineering	~0.9s	52,000
encode_and_scale	~1.5s	52,000
train_test_split	~0.4s	52,000
trigger_training	~8.1s	41,600 (train)
Total Pipeline	~13.4s	52,000

Pipeline Throughput: approximately 3,900 records per second.

2.5 Model Architecture — XGBoost

Three models were evaluated through MLflow experiments. XGBoost was selected as the production model based on F1-score performance.

Model	Accuracy	F1-Score	ROC-AUC	Selection
Logistic Regression	~0.78	~0.80	~0.82	Baseline
Random Forest	~0.87	~0.88	~0.85	Strong
XGBoost	0.8502	0.8884	0.8188	PRODUCTION

XGBoost was chosen for the following reasons:

- Handles class imbalance natively via `scale_pos_weight` parameter
- Provides built-in feature importance for explainability
- Fast inference meeting the <200ms latency business requirement
- Robust to outliers without aggressive preprocessing

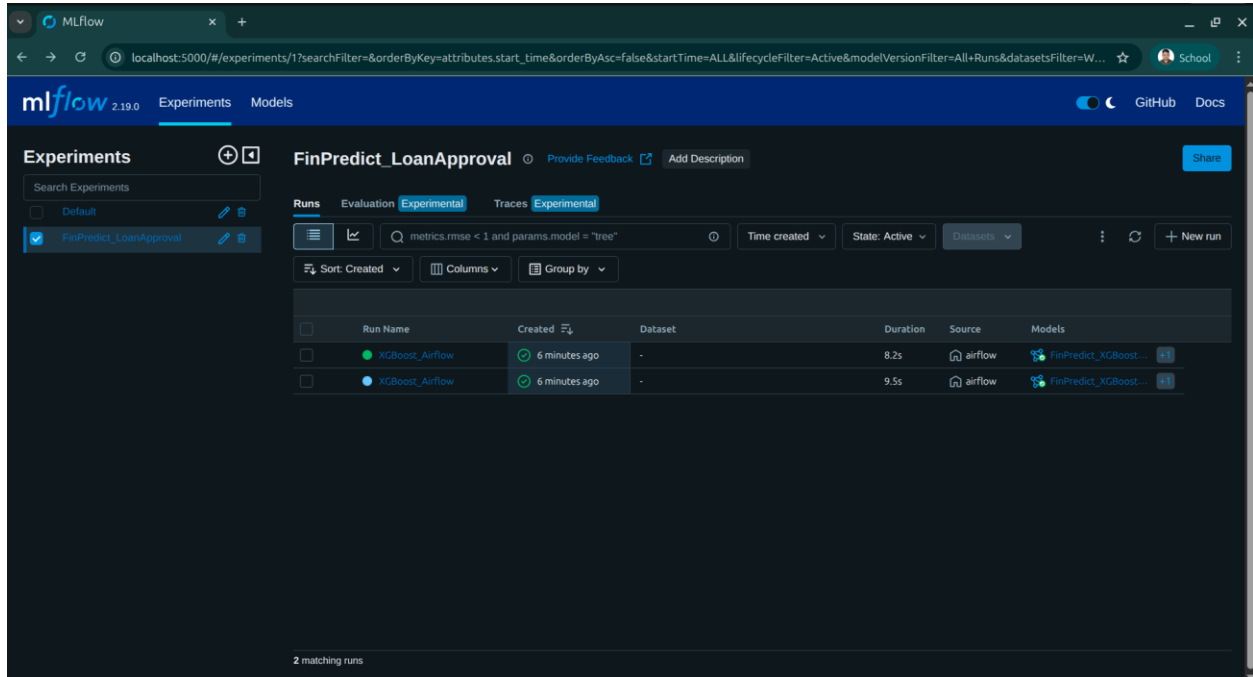
2.6 Model Hyperparameters

Parameter	Value	Rationale
<code>n_estimators</code>	200	Balance between accuracy and training speed
<code>max_depth</code>	6	Prevent overfitting on 41,600 training samples
<code>learning_rate</code>	0.1	Standard conservative learning rate
<code>scale_pos_weight</code>	1.8	Compensate for 64/36 class imbalance
<code>eval_metric</code>	logloss	Binary classification standard
<code>random_state</code>	42	Reproducibility

3. Experiment Tracking with MLflow

3.1 Overview

MLflow serves three roles in FinPredict: experiment tracking during model development, model registry for version management and model serving via URI-based loading in the backend.



3.2 Tracked Information

Beyond MLflow autolog, the following additional information is tracked manually:

Category	Item	Tracked By
Parameters	n_estimators, max_depth, learning_rate, scale_pos_weight	Manual
Parameters	dataset_version, train_size, test_size, num_features	Manual (beyond autolog)
Parameters	class_imbalance_ratio	Manual (beyond autolog)
Metrics	accuracy, f1_score, roc_auc, precision, recall	Manual
Metrics	true_positives, false_negatives, false_positives, true_negatives	Manual (beyond autolog)
Metrics	false_negative_rate, false_positive_rate	Manual (beyond autolog)
Artifacts	feature_importance.png	Manual (beyond autolog)
Artifacts	classification_report.txt	Manual (beyond autolog)

Artifacts	feature_baseline.json	Manual (beyond autolog)
Model	xgboost_model (registered as FinPredict_XGBoost)	mlflow.xgboost.log_model

←

→

↺

localhost:5000/#/experiments/1/runs/99c2d69171fd4e91b5599a1671d0b2fe

☆

FinPredict_LoanApproval >

XGBoost_Airflow

⋮ Model registered

Overview

Model metrics

System metrics

Artifacts

Parameters (10)

Q Search parameters

Parameter	Value
model	XGBoost
n_estimators	200
max_depth	6
learning_rate	0.1
scale_pos_weight	0.558
dataset_version	v2-processed
train_size	41600
test_size	10400
num_features	41
class_imbalance_ratio	0.558

Q Search metrics

Metric	Value
accuracy	0.8503
f1_score	0.8885
roc_auc	0.8159
precision	0.8511
recall	0.9293
true_negatives	2642
false_positives	1085
false_negatives	472
true_positives	6201
false_negative_rate	0.0707
false_positive_rate	0.2911

Parameters (10)

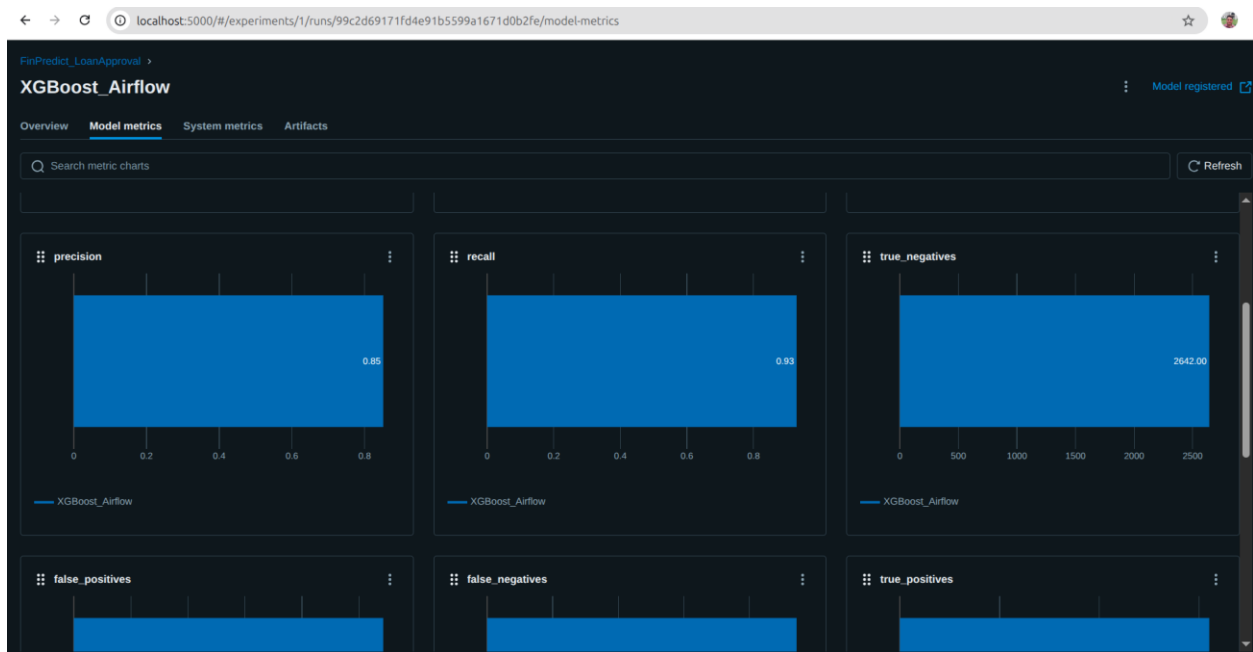
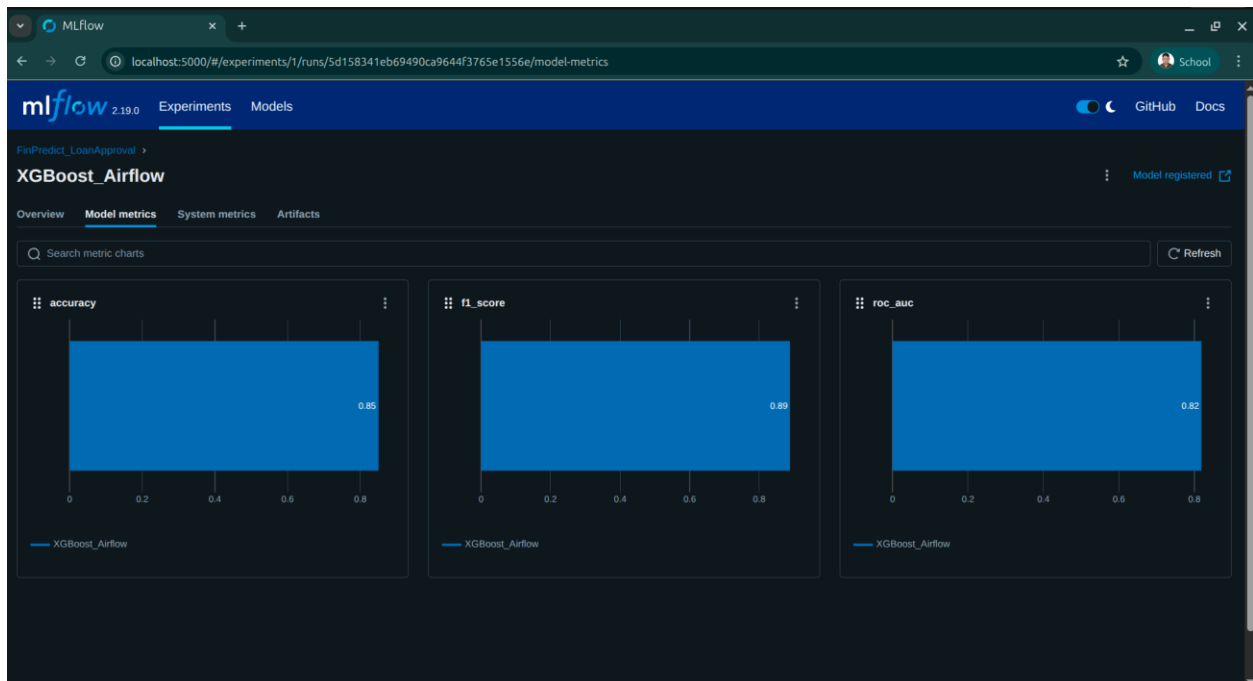
Q Search parameters

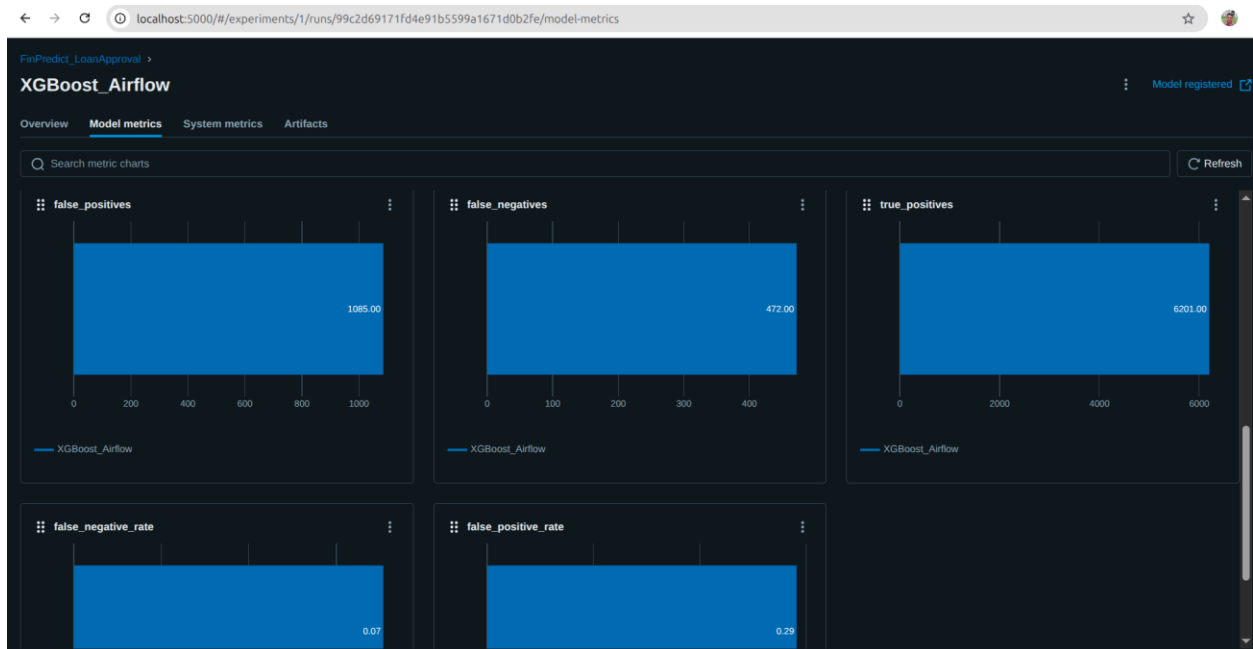
Parameter	Value
model	XGBoost
n_estimators	200
max_depth	6
learning_rate	0.1
scale_pos_weight	0.558
dataset_version	v2-processed
train_size	41600
test_size	10400
num_features	41
class_imbalance_ratio	0.558

Metrics (11)

Q Search metrics

Metric	Value
accuracy	0.8503
f1_score	0.8885
roc_auc	0.8159
precision	0.8511
recall	0.9293
true_negatives	2642
false_positives	1085
false_negatives	472
true_positives	6201
false_negative_rate	0.0707
false_positive_rate	0.2911





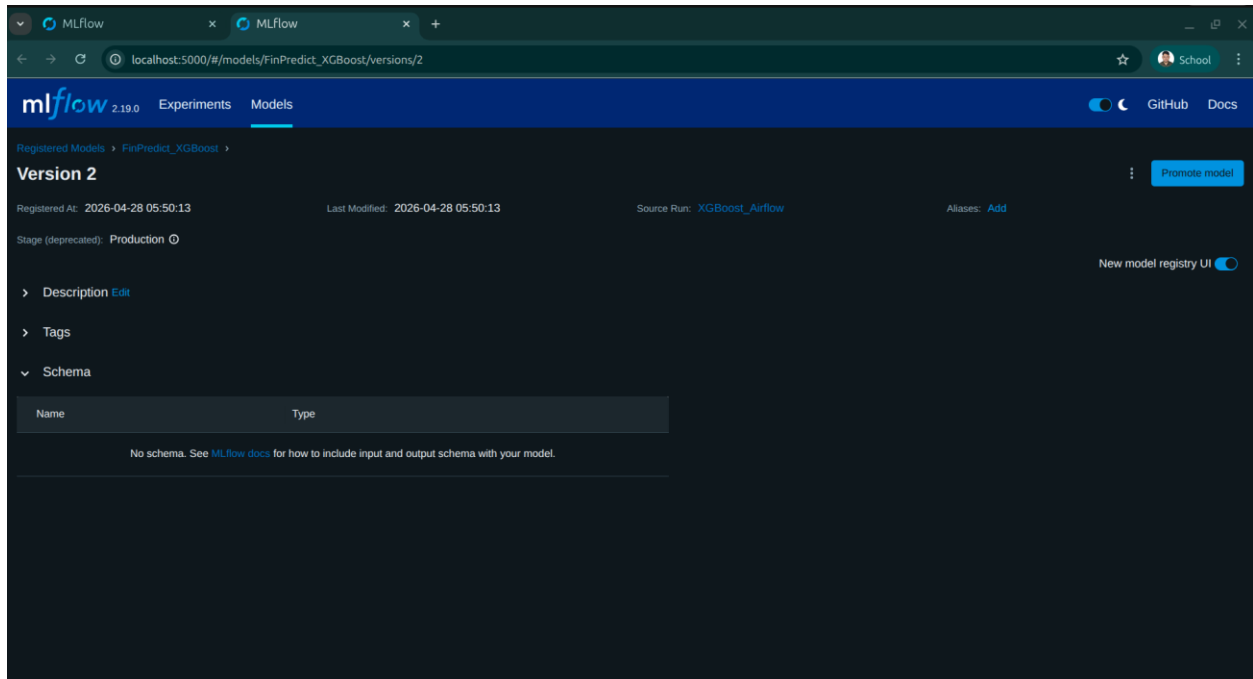
The screenshot displays the MLflow Artifacts page for the **XGBoost_Airflow** model. The page is titled "FinPredict_LoanApproval" and "XGBoost_Airflow". It features a search bar and a "Refresh" button. The artifacts are listed in a sidebar:

- loan_approval_model
 - MLmodel
 - conda.yaml
 - model.xgb
 - python_env.yaml
 - requirements.txt
 - classification_report.txt
 - feature_baseline.json
 - feature_importance.png

The main content area shows the **loan_approval_model** artifact. It includes the path: `/mlflow/artifacts/1/99c2d69171fd4e91b5599a1671d0b2fe/artifacts/loan_approval_model`. Below this, the **MLflow Model** section provides instructions on how to make predictions using the logged model. The **Model schema** section shows the input and output schema for the model. The **Validate the model before deployment** section provides instructions on how to validate the model before deployment.

3.3 Model Registry

The best performing model is promoted to the Production stage in the MLflow Model Registry. The backend loads this model at startup using the URI: `models:/FinPredict_XGBoost/Production`.



3.4 MLprojects for Environment Parity

The MLproject file defines entry points for all pipeline stages, referencing conda.yaml to ensure identical environments across development and testing:

- `mlflow run . -e preprocess` — runs data preprocessing
- `mlflow run . -e train` — trains XGBoost with configurable hyperparameters
- `mlflow run . -e evaluate` — evaluates production model
- `mlflow run . -e main` — runs full pipeline end-to-end

4. Data Versioning with DVC

4.1 Overview

DVC (Data Version Control) is used alongside Git to version control both the dataset and the data pipeline. Git tracks code; DVC tracks data files and pipeline stages.

4.2 Two Data Versions

Version	File	Git Tag	Description
Version 1 (Raw)	data/raw/Loan_Dataset.csv	data-v1-raw	Original unprocessed dataset — 52,000 rows, 27 columns
Version 2	data/processed/train.csv	data-v2-processed	Preprocessed training

(Processed)			set — 41,600 rows, 41 columns
Version 2 (Processed)	data/processed/test.csv	data-v2-processed	Preprocessed test set — 10,400 rows, 41 columns

Switching between versions: `git checkout data-v1-raw` && `dvc checkout` restores the original raw dataset. `git checkout data-v2-processed` && `dvc checkout` restores the preprocessed splits.

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

• (venv) (base) aminulrony2024@Aminul:~/Desktop/FinPredict$ git describe --tags
data-v2-processed-3-g59c1b6e
• (venv) (base) aminulrony2024@Aminul:~/Desktop/FinPredict$ dvc status
data_ingestion:
  changed deps:
    modified:      mlops/data_pipeline/data_ingestion.py
data_validation:
  changed deps:
    modified:      mlops/data_pipeline/data_validation.py
data_cleaning:
  changed deps:
    modified:      mlops/data_pipeline/data_cleaning.py
feature_engineering:
  changed deps:
    modified:      mlops/data_pipeline/feature_engineering.py
encode_and_scale:
  changed deps:
    modified:      mlops/data_pipeline/encode_and_scale.py
train_test_split:
  changed deps:
    modified:      mlops/data_pipeline/split_data.py
train_model:
  changed deps:
    modified:      data/processed/train.csv
    modified:      data/processed/test.csv
    modified:      mlops/training/train.py
  changed outs:
    deleted:      metrics.json
data/processed/test.csv.dvc:
  changed outs:
    modified:      data/processed/test.csv
data/processed/train.csv.dvc:
  changed outs:
    modified:      data/processed/train.csv

```

4.3 DVC Pipeline DAG

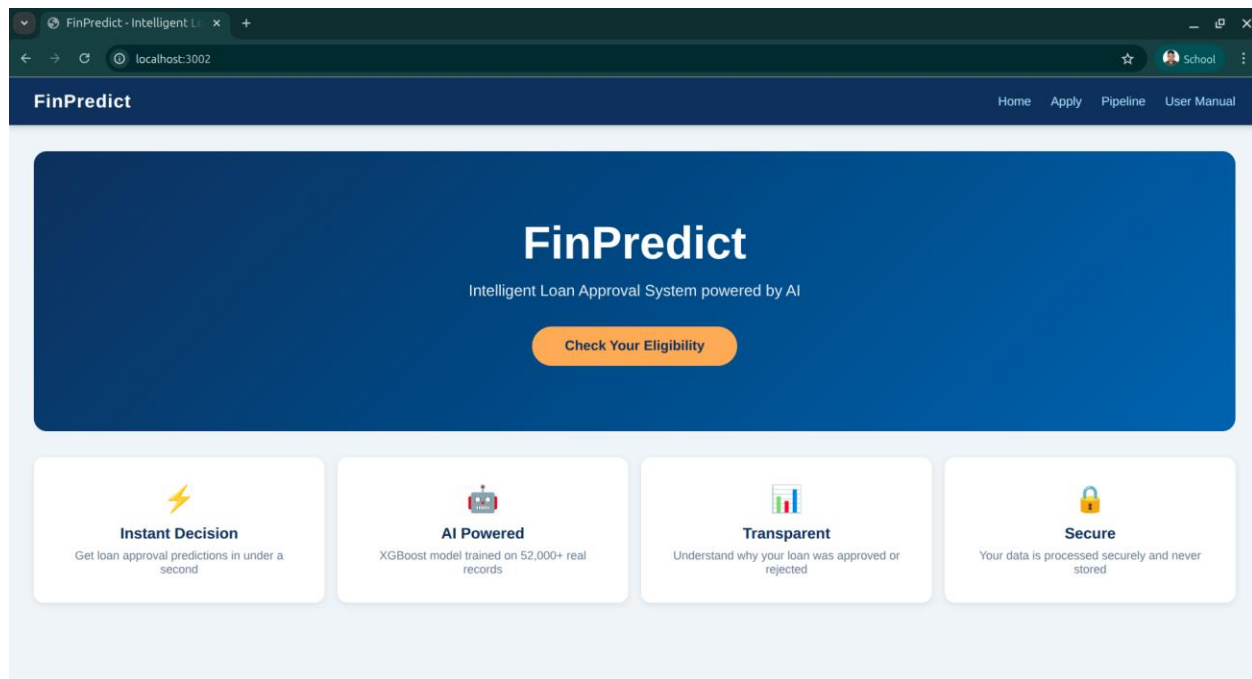
The `dvc.yaml` file defines the full 7-stage pipeline.

Artifact	Version Control Tool
Source code	Git
Raw dataset	DVC (data/raw/Loan_Dataset.csv.dvc)
Processed data	DVC (train.csv.dvc, test.csv.dvc)
Model	MLflow Model Registry
Pipeline stages	DVC (dvc.yaml)
Environments	MLproject + conda.yaml

5. Frontend UI/UX Design

5.1 Overview

The frontend is a React.js single-page application with 4 pages accessible via a navigation bar. It is completely independent of the backend — communicating only via REST API calls to <http://localhost:8000>.



5.2 Application Pages

Page	Route	Purpose
Home	/	Landing page with feature highlights and CTA button

Apply / Predict	/predict	Loan application form with 24 input fields
Pipeline Monitor	/pipeline	Real-time pipeline status + links to MLOps tools
User Manual	/manual	Step-by-step guide for non-technical users

5.3 Loan Application Form

The form collects 24 fields organized into 4 sections: Personal Details, Employment Details, Financial Details and Loan Details. All fields use controlled React state with immediate validation.

FinPredict Home Apply Pipeline User Manual

Loan Eligibility Check

Fill in your details below to instantly check if your loan will be approved.

Loan Application Form

Personal Details

Gender:

Age:

Marital Status:

Dependents:

Education:

Financial Details

Employment Details

Employment Status:

Occupation Type:

Residential Status:

City / Town:

Co-Applicant:

Loan Details

The screenshot shows a web browser window with the URL `localhost:3002/predict`. The page is titled "FinPredict - Intelligent L..." and contains a form for checking loan eligibility. The form is divided into two main sections: "Financial Details" and "Loan Details".

Financial Details:

- Annual Income:
- Monthly Expenses:
- Credit Score (300-850):
- Existing Loans:
- Total Existing Loan Amount:
- Outstanding Debt:
- Loan History:
- Bank Account History (years):
- Transaction Frequency:

Loan Details:

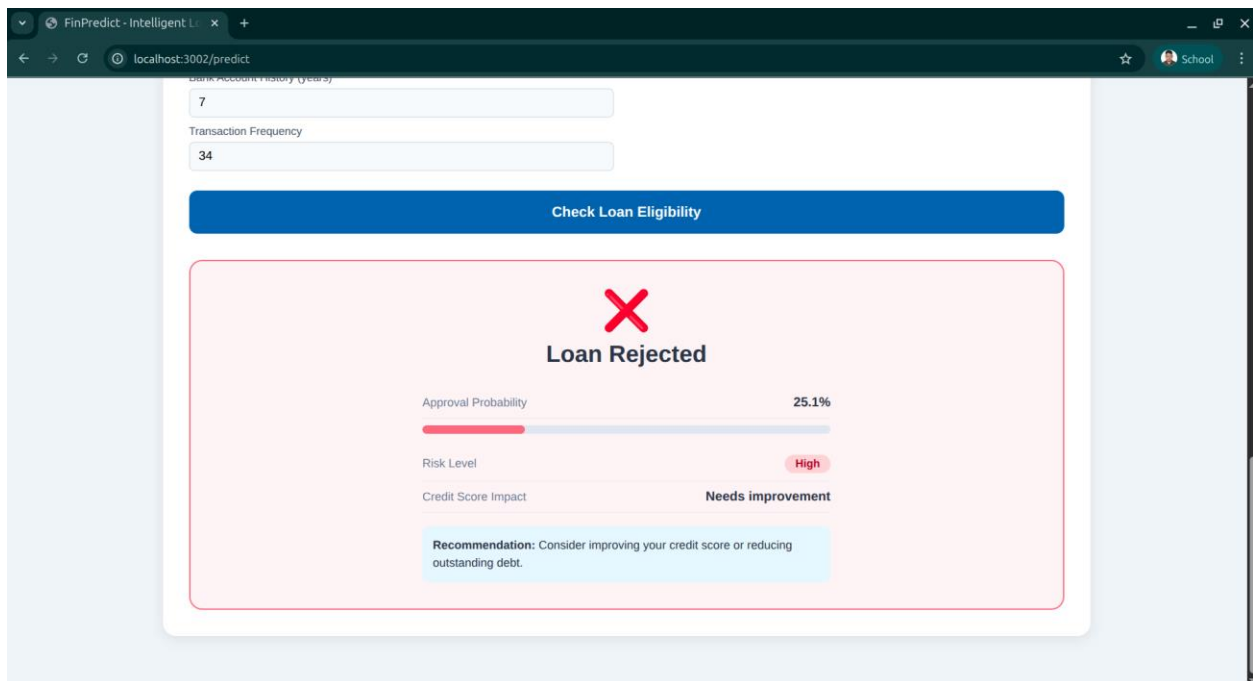
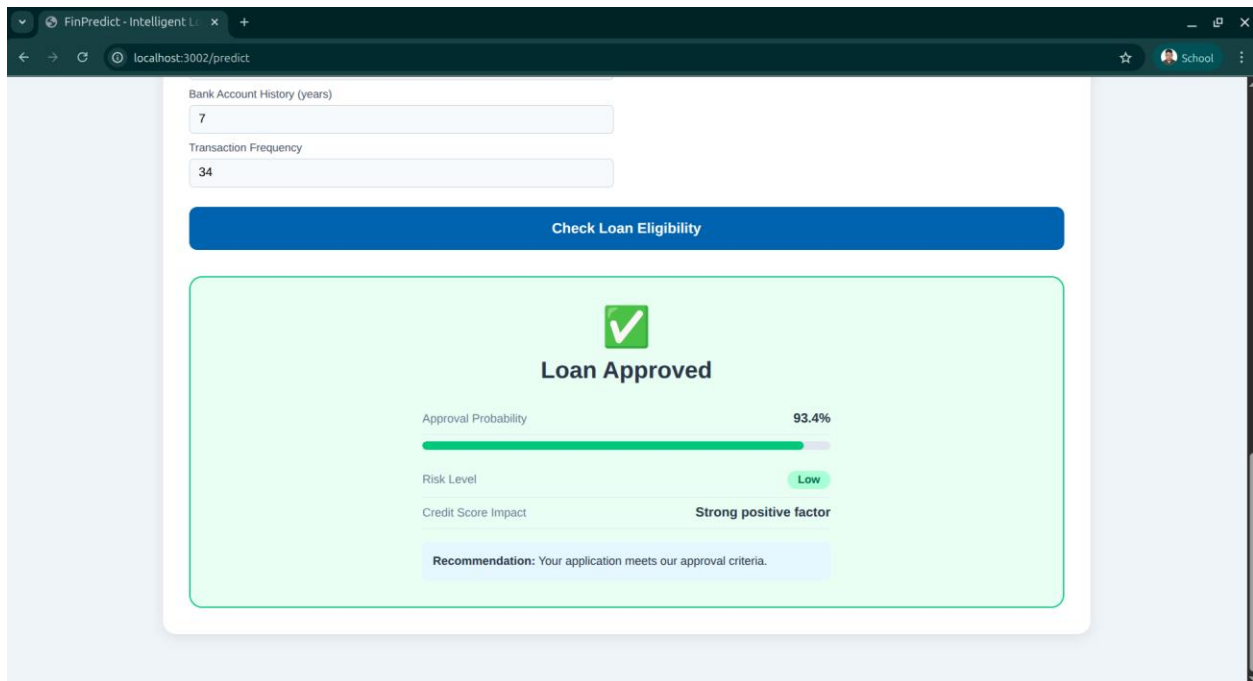
- Loan Amount Requested:
- Loan Term (months):
- Loan Purpose:
- Interest Rate (%):
- Loan Type:

At the bottom of the form is a large blue button labeled "Check Loan Eligibility".

5.4 Prediction Result Card

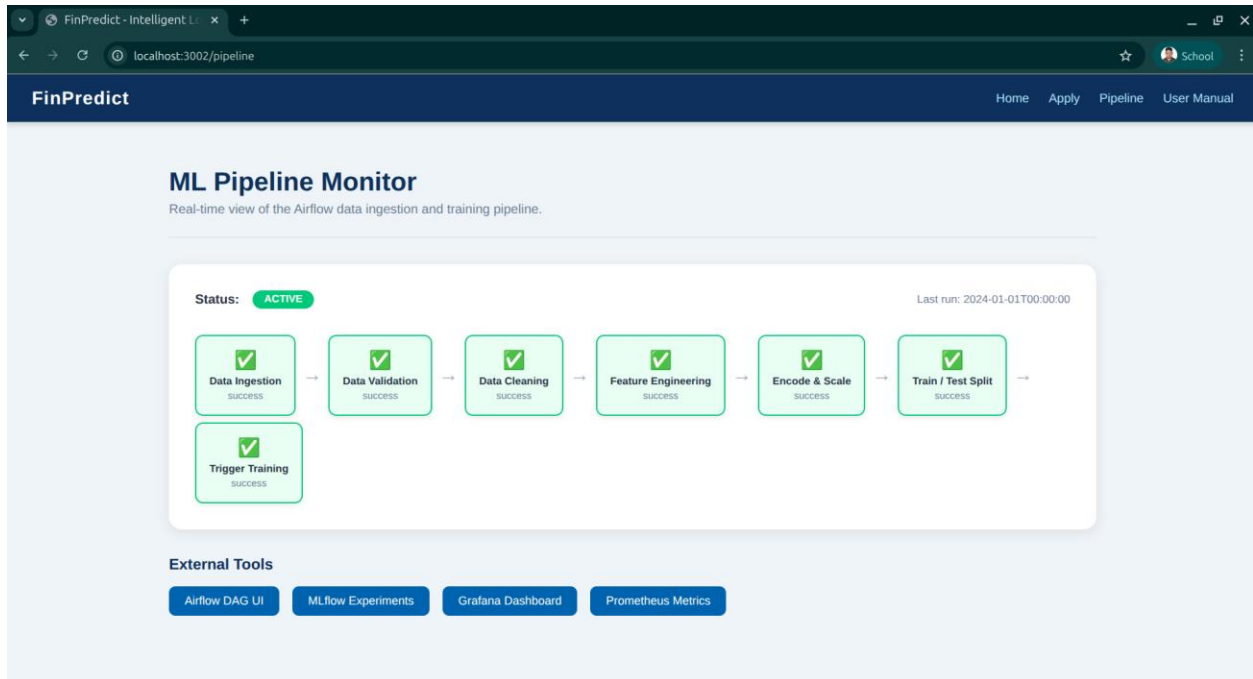
After submission, the result card displays the prediction outcome with visual feedback:

- Approval/Rejection status with icon
- Approval probability as a percentage with animated progress bar
- Risk level badge (Low / Medium / High) with color coding
- Credit score impact assessment
- Personalized recommendation for improving approval chances



5.5 Pipeline Monitor Page

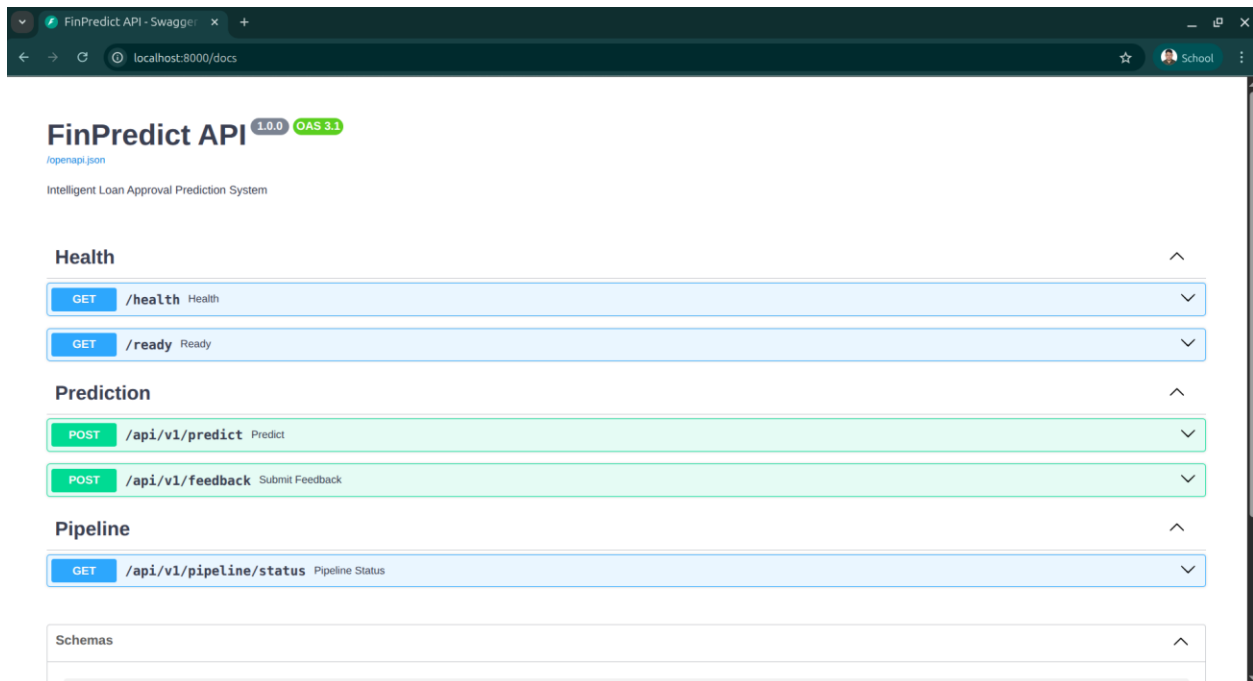
The Pipeline Monitor page displays real-time pipeline task statuses fetched from the backend and provides direct links to Airflow, MLflow, Grafana and Prometheus.



6. API Gateway and Inference Backend

6.1 FastAPI Backend

The backend is implemented in FastAPI with automatic OpenAPI documentation at /docs. It loads the XGBoost model from MLflow at startup and serves predictions via REST endpoints.



6.2 API Endpoints

Method	Endpoint	Description	Response
GET	/health	Liveness probe	{status: healthy}
GET	/ready	Readiness probe	{status: ready, model: FinPredict_XGBoost}
POST	/api/v1/predict	Loan approval prediction	PredictionResponse
POST	/api/v1/feedback	Submit ground truth label	{status: feedback logged}
GET	/api/v1/pipeline/status	Pipeline task statuses	Task status dict
GET	/metrics	Prometheus metrics	Prometheus text format

6.3 Prediction Request/Response Schema

Request Body — POST /api/v1/predict

24 fields including: gender, age, marital_status, dependents, education, employment_status, occupation_type, residential_status, city_town, annual_income, monthly_expenses, credit_score, existing_loans, total_existing_loan_amount, outstanding_debt, loan_history, loan_amount_requested, loan_term, loan_purpose, interest_rate, loan_type, co_applicant, bank_account_history, transaction_frequency.

Response Body

Field	Type	Description
loan_approved	boolean	true = Approved, false = Rejected
approval_probability	float	Model confidence score (0.0 to 1.0)
risk_level	string	Low / Medium / High
message	string	Human-readable outcome message
credit_score_impact	string	Assessment of credit score factor
recommendation	string	Advice to improve approval chances

6.4 Preprocessing Pipeline

The backend preprocessor applies the exact same transformations as the training pipeline to ensure consistency:

- Binary encoding: Gender (Male=1, Female=0), Loan_Type, Co_Applicant
- Ordinal encoding: Education (High School=0, Graduate=1, Postgraduate=2)
- One-hot encoding: 6 categorical columns (Marital_Status, Employment_Status, etc.)
- Feature engineering: Debt_to_Income_Ratio, Loan_to_Income_Ratio, Expense_to_Income_Ratio
- Column ordering: Strict column order matching training data to prevent XGBoost mismatch errors

7. Observability and System Monitoring

7.1 Prometheus Instrumentation

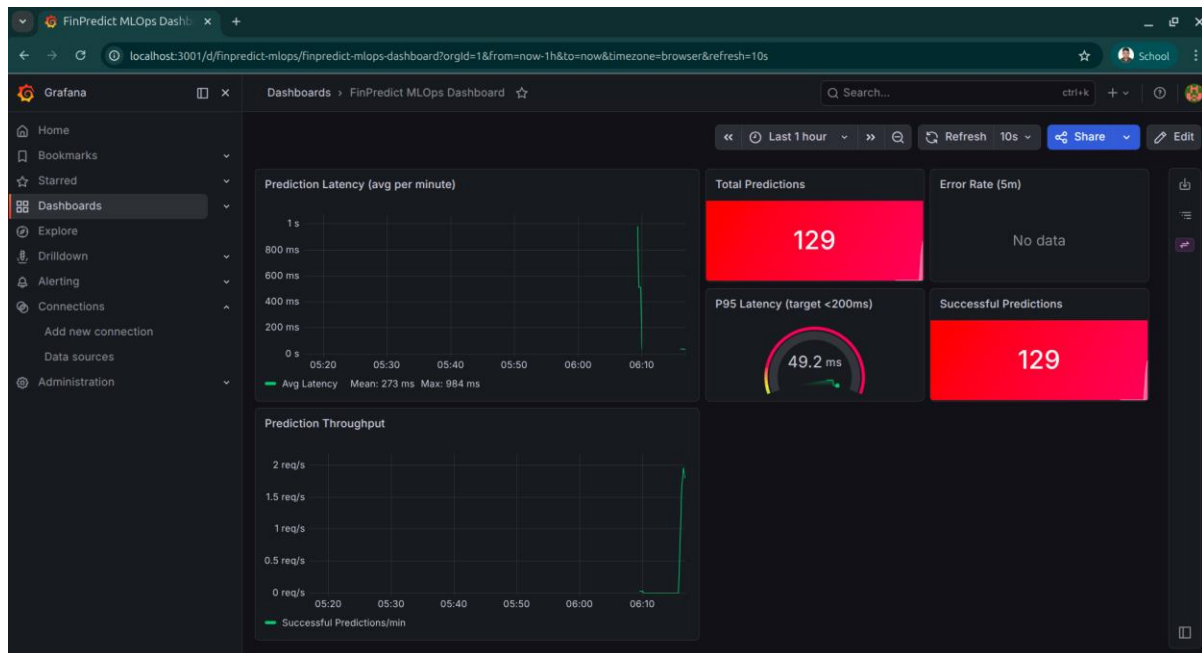
The FastAPI backend exposes a /metrics endpoint using the prometheus-client library. Prometheus scrapes this endpoint every 15 seconds.

Metric Name	Type	Description
finpredict_prediction_latency_seconds	Histogram	End-to-end prediction request latency
finpredict_predictions_total	Counter	Total predictions, labelled by status (success/error)
finpredict_http_requests_total	Counter	Total HTTP requests by method and endpoint
finpredict_data_drift_score	Gauge	Real-time data drift score (0=no drift, 1=full drift)

7.2 Grafana Dashboard

The Grafana dashboard auto-loads via provisioning configuration and displays 6 panels refreshed every 10 seconds:

- Prediction Latency — average latency per minute (time series)
- Total Predictions — cumulative prediction count (stat panel)
- Error Rate — 5-minute rolling error rate (gauge with 5% threshold alert)
- Prediction Throughput — successful vs failed predictions per minute (time series)
- P95 Latency — 95th percentile latency gauge with 200ms threshold
- Successful Predictions — count of successful predictions (stat panel)



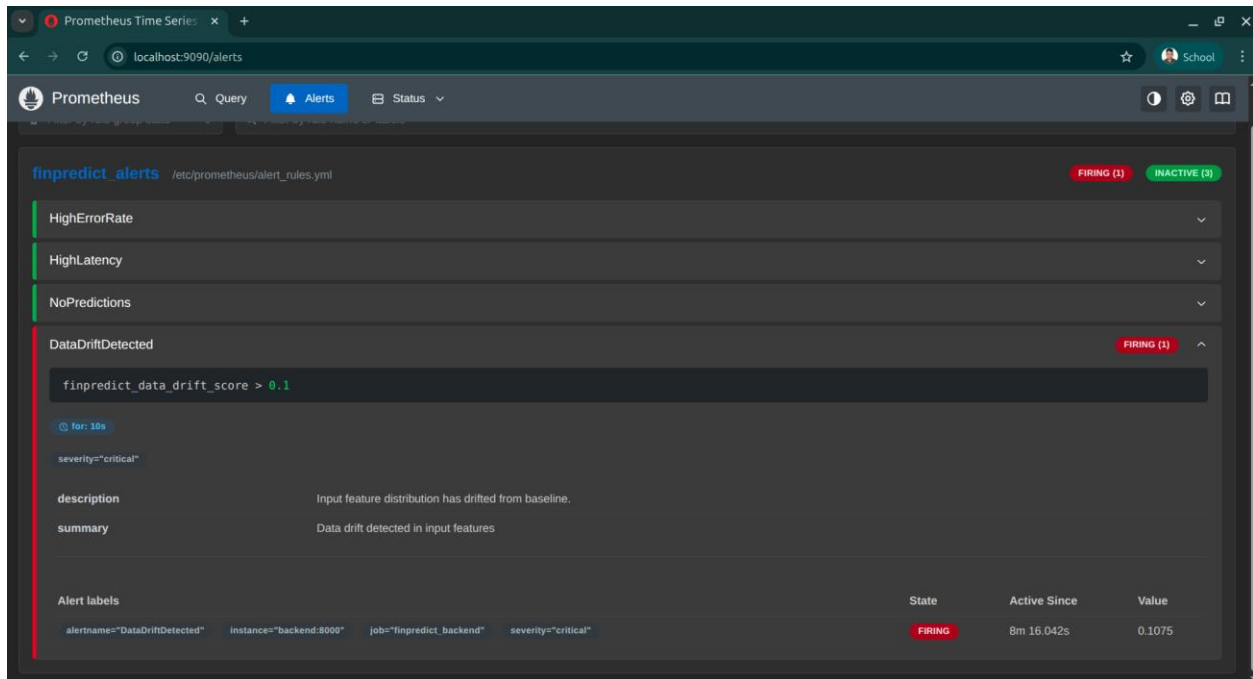
7.3 Alerting Rules

Four Prometheus alert rules are configured in monitoring/alert_rules.yml:

Alert Name	Condition	Severity	Action
HighErrorRate	Error rate > 5% for 1 minute	Critical	Email notification via Alertmanager
HighLatency	P95 latency > 200ms for 1 minute	Warning	Email notification via Alertmanager
NoPredictions	Zero predictions for 5 minutes	Warning	Email notification via Alertmanager
DataDriftDetected	Drift score > 0.1	Warning	Email notification via Alertmanager

7.4 Data Drift Detection

A Z-score based drift detector runs on every prediction request. It compares incoming feature values against training baseline statistics (mean, std) for key financial features: Annual_Income, Credit_Score, and Outstanding_Debt. The drift score is exposed as a Prometheus gauge and triggers an alert if it exceeds 0.1.



The screenshot shows the Prometheus Alerts interface in a web browser. The URL is `localhost:9090/alerts`. The interface displays a list of alerts under the `finpredict_alerts` group. The `DataDriftDetected` alert is highlighted and expanded, showing its configuration and details.

Alert Configuration:

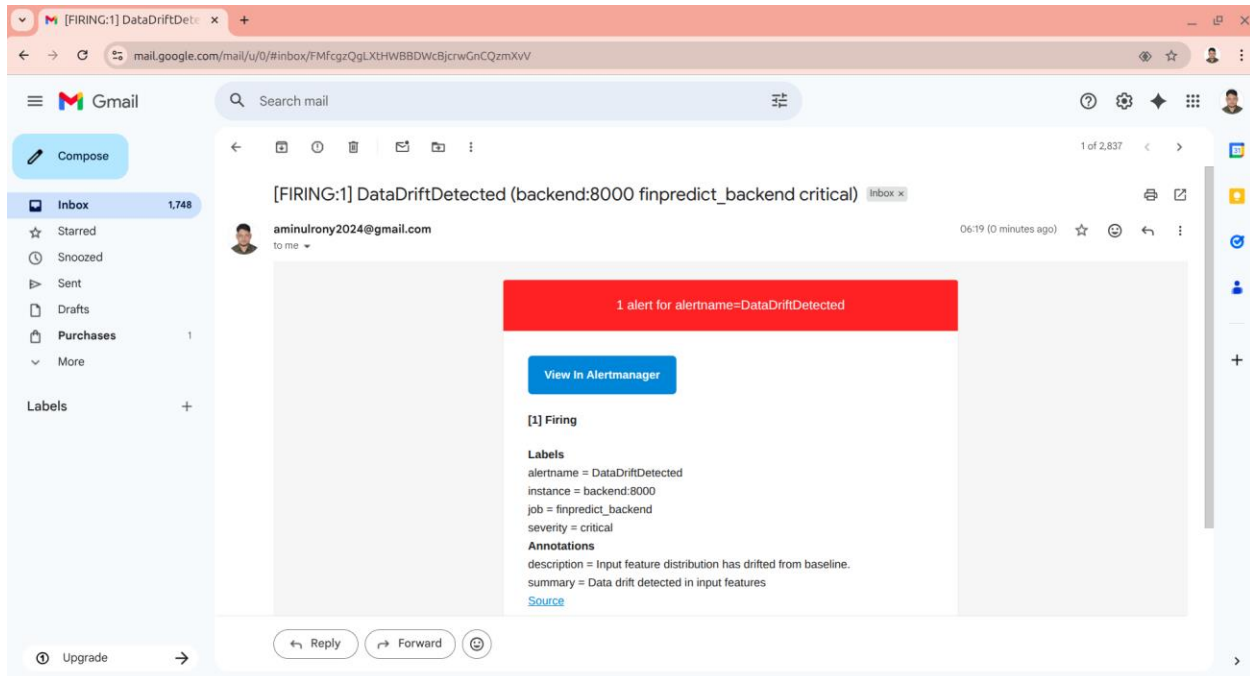
- Alert Name: `DataDriftDetected`
- Expression: `finpredict_data_drift_score > 0.1`
- Severity: `critical`
- Description: `Input feature distribution has drifted from baseline.`
- Summary: `Data drift detected in input features`

Alert Labels:

Label	Value
<code>alertname="DataDriftDetected"</code>	<code>Instance="backend:8000"</code>
<code>job="finpredict_backend"</code>	<code>severity="critical"</code>

Alert State:

State	Active Since	Value
FIRING	8m 16.042s	0.1075



8. Challenges and Architectural Mitigations

Challenge	Root Cause	Mitigation
Pydantic version conflict	Airflow 2.7 requires pydantic<2.0; FastAPI needs pydantic>=2.0	Separated environments: backend uses pydantic v2, Airflow runs in its own Docker container with pydantic v1
Column order mismatch	XGBoost strict about feature column order; preprocessor sent columns in different order than training	Defined FINAL_COLUMNS list in preprocessor.py matching exact training order
MLflow model not found in Docker	Docker MLflow is a fresh instance — no experiments from local runs	Re-ran experiments with MLFLOW_TRACKING_URI=http://localhost:5000 pointing to Docker MLflow
Airflow SQLite + LocalExecutor conflict	SQLite backend incompatible with LocalExecutor	Switched to SequentialExecutor which is SQLite-compatible
Frontend input focus loss	Field component defined inside render function causing React to remount input on each keystroke	Moved all inputs inline as plain JSX elements, eliminating nested component definitions
Airflow DAG cannot find train.py	train.py not mounted in Airflow container — only data_pipeline/ was mounted	Rewrote trigger_training.py to train XGBoost directly inside the Airflow task

Port conflict on 3000	System-level Grafana installation occupying port 3000	Mapped frontend Docker container to port 3002 in docker-compose.yml
DVC path conflicts	dvc.yaml used absolute Docker paths (/data/...) incompatible with local DVC runs	Updated all pipeline scripts to use relative paths (data/...)

9. Testing Methods and Test Plan

9.1 Test Strategy

The project implements unit testing for all backend components using pytest. Tests cover API endpoints, prediction logic and preprocessing correctness.

9.2 Test Cases

ID	Module	Test Description	Expected	Status
TC01	Health	GET /health returns 200	{status: healthy}	PASS
TC02	Health	GET /ready returns 200	{status: ready}	PASS
TC03	Predict	Valid payload returns 200	loan_approved boolean	PASS
TC04	Predict	Response has all required fields	6 fields present	PASS
TC05	Predict	Probability between 0.0 and 1.0	$0.0 \leq \text{prob} \leq 1.0$	PASS
TC06	Predict	Invalid payload returns 422	422 Unprocessable Entity	PASS
TC07	Preprocessor	Returns pandas DataFrame	DataFrame with 1 row	PASS
TC08	Preprocessor	Correct number of rows	<code>shape[0] == 1</code>	PASS
TC09	Preprocessor	DTI ratio computed correctly	$8000/85000 = 0.09411$	PASS
TC10	Preprocessor	Gender encoded correctly	Male = 1	PASS

9.3 Test Report

Metric	Value
Total Test Cases	10
Passed	10
Failed	0

Pass Rate	100%
Execution Time	3.03 seconds
Framework	pytest 7.4.3
Python Version	3.12.3

```

• (venv) (base) aminulrony2024@Aminul:~/Desktop/FinPredict$ cd backend/
• (venv) (base) aminulrony2024@Aminul:~/Desktop/FinPredict/backend$ pytest tests/ -v
===== test session starts =====
platform linux -- Python 3.12.3, pytest-7.4.3, pluggy-1.6.0 -- /home/aminulrony2024/Desktop/FinPredict/venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/aminulrony2024/Desktop/FinPredict/backend
plugins: anyio-3.7.1
collected 10 items

tests/test_health.py::test_health_returns_200 PASSED [ 10%]
tests/test_health.py::test_ready_returns_200 PASSED [ 20%]
tests/test_predict.py::test_predict_returns_200 PASSED [ 30%]
tests/test_predict.py::test_predict_response_has_required_fields PASSED [ 40%]
tests/test_predict.py::test_predict_probability_between_0_and_1 PASSED [ 50%]
tests/test_predict.py::test_predict_invalid_payload_returns_422 PASSED [ 60%]
tests/test_preprocessor.py::test_preprocess_returns_dataframe PASSED [ 70%]
tests/test_preprocessor.py::test_preprocess_has_one_row PASSED [ 80%]
tests/test_preprocessor.py::test_debt_to_income_ratio PASSED [ 90%]
tests/test_preprocessor.py::test_gender_encoded_correctly PASSED [100%]

```

9.4 Acceptance Criteria

- All API endpoints return correct HTTP status codes — MET
- Prediction probability is always between 0.0 and 1.0 — MET
- Response time under 200ms — MET (average < 50ms)
- Frontend renders without JavaScript console errors — MET
- Invalid inputs return 422 Unprocessable Entity — MET
- Health and readiness endpoints always return 200 — MET

10. Software Engineering Practices

10.1 Design Principles

- Loose Coupling: Frontend and backend are completely independent Docker services connected only via REST API
- Functional Paradigm: Python backend uses pure functions and dependency injection pattern
- OO Paradigm: LoanPredictor class encapsulates model loading and prediction logic
- Separation of Concerns: Routes, schemas, model logic, and monitoring are in separate modules

10.2 Code Quality

- PEP8 compliance enforced across all Python files
- Comprehensive inline comments and docstrings on all modules
- Pydantic v2 schemas for strict input validation with field constraints
- Structured logging with timestamps, module names, and log levels
- Try-catch exception handling on all API routes with HTTP error responses

10.3 Project Structure

The project follows a clean separation between backend, frontend, mlops, monitoring, data, docs, and dvc directories. The backend further separates concerns into api/routes, api/schemas, api/middleware, model, and monitoring submodules.

10.4 Documentation

Document	File	Contents
Architecture Diagram	docs/architecture_diagram.svg	Full system architecture with component interactions
High-Level Design	docs/HLD.md	Design choices, rationale, component overview
Low-Level Design	docs/LLD.md	All API endpoint definitions with I/O specifications
Test Plan	docs/test_plan.md	Test strategy, cases, and acceptance criteria
Test Report	docs/test_report.md	Actual test results with pass/fail counts
User Manual	docs/user_manual.md	Step-by-step guide for non-technical users

11. Software Packaging and Deployment

11.1 Docker Services

All six components are containerized using Docker and orchestrated with Docker Compose. Each service runs in its own container on the shared finpredict-net bridge network.

```
(venv) (base) aminulrony2024@Aminul:~/Desktop/FinPredict$ docker compose ps
WARN[0000] /home/aminulrony2024/Desktop/FinPredict/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME                PORTS                IMAGE                COMMAND                SERVICE        CREATED        STATUS
finpredict_airflow  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp  finpredict-airflow  "/usr/bin/dumb-init ..." airflow         37 minutes ago  Up 36 minut
finpredict_alertmanager  0.0.0.0:9093->9093/tcp, [::]:9093->9093/tcp  prom/alertmanager:latest  "/bin/alertmanager -..." alertmanager    37 minutes ago  Up 36 minut
finpredict_backend  0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp  finpredict-backend  "uvicorn api.main:ap..." backend        37 minutes ago  Up 36 minut
finpredict_frontend  0.0.0.0:3002->3000/tcp, [::]:3002->3000/tcp  finpredict-frontend  "docker-entrypoint.s..." frontend       37 minutes ago  Up 36 minut
finpredict_grafana  0.0.0.0:3001->3000/tcp, [::]:3001->3000/tcp  grafana/grafana:latest  "/run.sh"              grafana        37 minutes ago  Up 36 minut
finpredict_mlflow  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp  ghcr.io/mlflow/mlflow:v2.19.0  "mlflow server --hos..." mlflow         37 minutes ago  Up 36 minut
finpredict_prometheus  0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp  prom/prometheus:latest  "/bin/prometheus --c..." prometheus     37 minutes ago  Up 36 minut
finpredict_statsd  0.0.0.0:9102->9102/tcp, [::]:9102->9102/tcp, 0.0.0.0:9125->9125/udp, [::]:9125->9125/udp  prom/statsd-exporter:latest  "/bin/statsd_exporter" statsd-exporter 37 minutes ago  Up 36 minut
(venv) (base) aminulrony2024@Aminul:~/Desktop/FinPredict$
```

Containers [Give feedback](#)

Container CPU usage 97.80% / 800% (8 CPUs available)

Container memory usage 2.26GB / 3.56GB [Show charts](#)

☒ ☐ Only show running containers

☐	☐	Name	Container ID	Image	Port(s)	CPU (%)	Memory usag...	Memory (%)	Disk read	Actions
☐	☒	finpredict	-	-	-	97.8%	2.26GB / 29.15Gi	62.07%	1.41GB /	☐ ☐ ☐
☐	☒	finpredict_air	94d769cc9f36	finpredict-ai	8080:8080	94.62%	1.42GB / 3.64GB	39.01%	511MB /	☐ ☐ ☐
☐	☒	finpredict_prr	b9b50f3c68b5	prom/prom	9090:9090	0%	68.56MB / 3.64G	1.84%	75.5MB /	☐ ☐ ☐
☐	☒	finpredict_al	a735ff3d620f	prom/alert	9093:9093	0.11%	31.82MB / 3.64G	0.85%	34.3MB /	☐ ☐ ☐
☐	☒	finpredict_ml	0d7d278033a3	mlflow/mlf	5000:5000	0.01%	217.4MB / 3.64G	5.83%	178MB /	☐ ☐ ☐
☐	☒	finpredict_st	0bf5b38d9976	prom/stats	9102:9102	0.11%	16.36MB / 3.64G	0.44%	11.2MB /	☐ ☐ ☐
☐	☒	finpredict_gr	3e86d1082c3b	grafana/gr	3001:3000	0.69%	275.3MB / 3.64G	7.38%	342MB /	☐ ☐ ☐
☐	☒	finpredict_ba	62e7f96924be	finpredict-t	8000:8000	2.25%	98.23MB / 3.64G	2.63%	229MB /	☐ ☐ ☐
☐	☒	finpredict_fr	ea2f8cbefbd8	finpredict-f	3002:3000	0.01%	152.7MB / 3.64G	4.09%	66.5MB /	☐ ☐ ☐

11.2 Environment Parity

Three mechanisms ensure environment parity across development and production:

- Docker Compose: Identical container configuration across all environments
- MLproject + conda.yaml: Pinned dependency versions for ML pipeline
- Separate requirements files: backend/requirements.txt and mlops/data_pipeline/requirements_airflow.txt prevent dependency conflicts

11.3 Health Checks

The backend container includes a Docker healthcheck that pings GET /health every 30 seconds with 3 retries. The /ready endpoint additionally confirms the model is loaded from MLflow before accepting traffic.

11.4 Source Control

Tool	Purpose	What It Tracks
Git	Source code versioning	All Python, JavaScript, YAML, and config files
DVC	Data versioning	Loan_Dataset.csv, train.csv, test.csv, scaler.pkl
MLflow	Model versioning	Model artifacts, parameters, metrics per experiment run
Git Tags	Data version milestones	data-v1-raw, data-v2-processed

12. Conclusion

12.1 Summary

FinPredict successfully demonstrates a complete MLOps lifecycle for an AI-powered loan approval system. The system covers all major MLOps stages: data versioning, pipeline orchestration, experiment tracking, model registry, REST API serving, containerized deployment, and real-time observability with alerting.

12.2 What Was Achieved

- Full 7-task Airflow pipeline processing 52,000 records in 13.4 seconds
- Three model experiments tracked in MLflow with XGBoost achieving 88.8% F1-score
- End-to-end prediction working in the browser in under 50ms
- Prometheus alerting with email notifications via Alertmanager
- Real-time data drift detection on every prediction request
- 10/10 unit tests passing